DATA SCIENCE LAB EXAM STUDY GUIDE

Program 05: K-Nearest Neighbors (KNN) Classifier

Source File: `e05_knn.py` | College Practical Exam Preparation

Table of Contents

Section 1: Program Identification and Aim — Page 1

Section 2: Prerequisites — Page 1

Section 3: Complete Exam-Ready Algorithm — Page 2

Section 4: Complete Theory (Formulas, Distance Metrics & Worked Example) — Page 3

Section 5: Complete Original Source Code & Block Walkthrough — Page 5

Section 6: Complete Function, Parameter and Syntax Reference — Page 6

Section 7: Step-by-Step Execution Walkthrough — Page 8

Section 8: Output and Interpretation — Page 9

Section 9: Exam Preparation (Record Answer, Question Bank, Model Answers) — Page 10

Page 11

C. Predict-the-Output Questions

1. **Question:** In `e05_knn.py`, what species is predicted for `sample = [[2, 2, 2, 2]]`?
   Answer: `setosa` (because all measurements are very small, lying firmly inside the Setosa cluster).
2. **Question:** When `test_size=0.25` is used on 150 Iris instances, how many samples are in `x_test`?
   Answer: 38 samples ($150 imes 0.25 = 37.5$, rounded to 38).

D. Identify-the-Function Questions

1. **Question:** Which class in `sklearn.neighbors` implements the K-Nearest Neighbors classifier?
   Answer: `KNeighborsClassifier(n_neighbors=...)`.
2. **Question:** Which function from `sklearn.metrics` computes the proportion of correct predictions?
   Answer: `accuracy_score(y_true, y_pred)`.
3. **Question:** Which function partitions dataset arrays into training and testing partitions?
   Answer: `train_test_split()` from `sklearn.model_selection`.

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Section 11: Common Mistakes and Troubleshooting — Page 13

Section 12: Memory Tricks and Last-Minute Rapid Revision — Page 14

Section 13: Cross-Program Comparison Table (Programs 5 to 10) — Page 15

Section 1: Program Identification and Aim

- **Program Number:** Program 05 (Machine Learning Series 1 of 6)
- **Exact Filename:** `e05_knn.py`
- **Actual Question from Source:** "Q05. Program to implement KNN Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm."
- **Aim of the Program:** To implement the K-Nearest Neighbors (KNN) supervised classification algorithm on the standard Iris flower dataset, evaluate classification accuracy on an unseen test set, classify an arbitrary new query sample, and visually display predicted classifications versus actual classifications using side-by-side scatter plots.
- **Problem Statement & Objectives:**
  1. Load the Iris dataset containing 150 flower samples across 3 species (Setosa, Versicolor, Virginica) described by 4 numerical morphological features.
  2. Partition the dataset into 75% training and 25% testing subsets using `train_test_split()`.
  3. Train a `KNeighborsClassifier` model with $k = 5$ neighbors.
  4. Predict classes for the test features and evaluate model accuracy via `metrics.accuracy_score()`.
  5. Classify a custom test sample `[[2, 2, 2, 2]]` and print its species name from `iris.target_names`.
  6. Plot side-by-side subplots comparing predicted class clusters against actual class clusters.
- **Input Data:** Scikit-Learn's built-in `load_iris()` dataset ($150 imes 4$ float feature matrix) and a single sample `[[2, 2, 2, 2]]`.
- **Target / Output:** Integer target classes ($0, 1, 2$) representing `'setosa'`, `'versicolor'`, `'virginica'`.
- **Expected Results:** Test accuracy $\ge 90\%$, new sample classified as `'setosa'`, and 2 side-by-side scatter subplots.

Section 2: Prerequisites

To master this program, a beginner must understand these concepts:

1. **Supervised Learning:** Machine learning where the training data includes both inputs (features X) and the correct answers (ground truth labels y).

2. **Classification vs. Regression:** In classification, the target y is a discrete category (flower species). In regression, the target is a continuous real number.
3. **Feature Matrix (X) & Target Vector (y):** In Iris, X is a 2D table of shape $(150, 4)$ containing measurements. y is a 1D array of shape $(150,)$ containing class integers ($0, 1, 2$).
4. **Training Set vs. Testing Set:** Training data (75%) teaches the model; testing data (25%) evaluates how well the model generalizes to unseen data.
5. **Euclidean Distance:** The straight-line distance between two points in n-dimensional space: $$d(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$
6. **Role of Scikit-Learn Modules:**
   - `sklearn.datasets.load_iris`: Supplies the pre-cleaned benchmark dataset.
   - `sklearn.model_selection.train_test_split`: Randomly partitions arrays into train and test subsets.
   - `sklearn.neighbors.KNeighborsClassifier`: Implements the distance-based KNN algorithm.
   - `sklearn.metrics`: Supplies evaluation metrics including `accuracy_score`.
   - `matplotlib.pyplot`: Generates graphical scatter subplots.

Section 3: Complete Exam-Ready Algorithm

GENERAL ALGORITHM (Exam Writing Format):

1. **START**
2. Import `load_iris` from `sklearn.datasets`, `train_test_split` from `sklearn.model_selection`, `KNeighborsClassifier` from `sklearn.neighbors`, `metrics` from `sklearn`, and `matplotlib.pyplot` as `plt`.
3. Load the benchmark Iris dataset into memory using `iris = load_iris()`.
4. Extract the feature measurements array x via `iris.data` and the ground-truth target class labels y via `iris.target`.
5. Partition the data into training (75%) and testing (25%) subsets using:
   `x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25)`.
6. Initialize the K-Nearest Neighbors classifier specifying 5 nearest neighbors:
   `model = KNeighborsClassifier(n_neighbors=5)`.
7. Fit the classifier using the training features and targets: `model.fit(x_train, y_train)`.
8. Generate class predictions for the unseen test features: `y_prediction = model.predict(x_test)`.
9. Compute and display the classification accuracy score: `metrics.accuracy_score(y_test, y_prediction)`.
10. Define a new 4-dimensional query sample: `sample = [[2, 2, 2, 2]]`.
11. Predict the class of the new sample using `model.predict(sample)[0]`, map the predicted integer index to its species name using `iris.target_names[pred]`, and display the result.
12. Activate Subplot 1 (`plt.subplot(1, 2, 1)`), scatter plot test feature column 0 vs column 1 colored by predicted classes (`c=y_prediction`), and set title to "Predicted classification" with axis labels "Sepal length" and "Sepal width".
13. Activate Subplot 2 (`plt.subplot(1, 2, 2)`), scatter plot test feature column 0 vs column 1 colored by actual true classes (`c=y_test`), and set title to "Actual classification" with axis labels "Sepal length" and "Sepal width".
14. Render and display both subplots simultaneously using `plt.show()`.
15. **STOP**

Section 4: Complete Theory

1. How the K-Nearest Neighbors Algorithm Works

KNN is a **non-parametric, instance-based, lazy learning algorithm**:

- **"Lazy Learner":** It does not construct an explicit internal mathematical model during `.fit()`. Instead, it simply stores all training vectors in memory. All computation occurs at inference time when `.predict()` is called.
- **"Instance-Based":** Predictions are derived directly from the proximity of stored training instances.
- **Classification Rule:** Given a query vector x_q, KNN calculates the distance between x_q and all stored training samples, selects the k closest neighbors, and assigns the class holding the **majority vote** among those k neighbors.

2. The Iris Dataset Anatomy

Feature Index	Feature Name	Measurement Unit	Typical Range
Column 0	Sepal Length	Centimeters (cm)	4.3 to 7.9 cm
Column 1	Sepal Width	Centimeters (cm)	2.0 to 4.4 cm
Column 2	Petal Length	Centimeters (cm)	1.0 to 6.9 cm
Column 3	Petal Width	Centimeters (cm)	0.1 to 2.5 cm

Target classes: $0 \implies$ **Setosa**, $1 \implies$ **Versicolor**, $2 \implies$ **Virginica**. 50 samples per class (150 total).

3. Step-by-Step Worked Numerical Example ($k=3$)

Suppose our model has stored 4 training points in 2D space, and we want to classify query point $Q = (2.0, 2.0)$:

Training Point (p)	Coordinates (x_1, x_2)	True Class	Euclidean Distance: $d = \sqrt{(x_1 - 2.0)^2 + (x_2 - 2.0)^2}$	Rank
A	$(1.5, 2.0)$	Class 0 (Setosa)	$\sqrt{(1.5-2)^2 + (2-2)^2} = \sqrt{0.25} = 0.50$	1 (Closest)
B	$(2.2, 2.1)$	Class 0 (Setosa)	$\sqrt{(2.2-2)^2 + (2.1-2)^2} = \sqrt{0.04+0.01} □pprox 0.22$	1 (Closest: 0.22)
C	$(3.5, 3.0)$	Class 1 (Versicolor)	$\sqrt{(3.5-2)^2 + (3.0-2)^2} = \sqrt{2.25+1.0} □pprox 1.80$	3
D	$(5.0, 4.0)$	Class 2 (Virginica)	$\sqrt{(5.0-2)^2 + (4.0-2)^2} = \sqrt{9+4} □pprox 3.61$	4

Voting ($k=3$): The 3 nearest neighbors are Point B (0.22, Class 0), Point A (0.50, Class 0), and Point C (1.80, Class 1).
Vote Tally: Class 0 has 2 votes; Class 1 has 1 vote. Majority winner $\implies$ **Class 0 (Setosa)**.

4. Impact of the Hyperparameter k

- **Small k (e.g. $k=1$):** Extremely flexible decision boundaries, highly sensitive to noise and outliers (High Variance / **Overfitting**).
- **Large k (e.g. $k=50$):** Overly smooth decision boundary, washes out small class clusters (High Bias / **Underfitting**).
- **Rule of Thumb:** $k = \sqrt{N}$ (often an odd integer to prevent ties in binary classification). In `e05_knn.py`, $k=5$ is chosen.

5. Advantages & Limitations

- **Advantages:** Simple to understand, zero training time (lazy), adapts naturally to multi-class problems.

- **Limitations:** Computationally expensive during prediction on large datasets ($O(N \cdot d)$ per query), sensitive to irrelevant features and unscaled data (curse of dimensionality).

Section 5: Complete Original Source Code & Walkthrough

Complete Verbatim Source Code (`e05_knn.py`)

```
# Exercise 5: KNN Algorithm

# Q05. Program to implement KNN Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm.

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
import matplotlib.pyplot as plt

iris = load_iris()

x = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25
)

model = KNeighborsClassifier(n_neighbors=5)

model.fit(x_train, y_train)

y_prediction = model.predict(x_test)

print("Accuracy = ", metrics.accuracy_score(y_test, y_prediction))

sample = [[2, 2, 2, 2]]

pred = model.predict(sample)[0]

print("New Sample belongs to ", iris.target_names[pred])


plt.subplot(1, 2, 1)

plt.scatter(
    x_test[:, 0],
    x_test[:, 1],
    c=y_prediction
)

plt.title("Predicted classification")
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")


plt.subplot(1, 2, 2)

plt.scatter(
    x_test[:, 0],
    x_test[:, 1],
    c=y_test
)

plt.title("Actual classification")
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")

plt.show()
```

Block-by-Block Execution Walkthrough

Block 1: Imports and Dataset Extraction (Lines 5–14)

- `from sklearn.datasets import load_iris`: Imports the Iris dataset loader.
- `from sklearn.model_selection import train_test_split`: Imports dataset partitioning utility.
- `from sklearn.neighbors import KNeighborsClassifier`: Imports KNN classification estimator.
- `from sklearn import metrics`: Imports accuracy evaluation functions.
- `import matplotlib.pyplot as plt`: Imports plotting tools.
- `iris = load_iris()`: Returns a dictionary-like `Bunch` object.
- `x = iris.data`: Matrix of shape $(150, 4)$ containing measurements.
- `y = iris.target`: Vector of shape $(150,)$ containing class integers $0, 1, 2$.

Block 2: Data Splitting, Model Training, and Evaluation (Lines 16–26)

- `train_test_split(x, y, test_size=0.25)`: Partitions 150 samples into 112 training samples and 38 testing samples.
- `model = KNeighborsClassifier(n_neighbors=5)`: Instantiates the model with $k=5$.
- `model.fit(x_train, y_train)`: Stores the 112 training points in memory.
- `y_prediction = model.predict(x_test)`: Finds 5 nearest neighbors in `x_train` for each of the 38 test vectors and assigns majority class.
- `metrics.accuracy_score(y_test, y_prediction)`: Compares 38 predicted classes to ground truth labels and returns accuracy float (e.g. `0.9736`).

Block 3: New Sample Classification (Lines 28–32)

- `sample = [[2, 2, 2, 2]]`: Defines a custom query flower with Sepal length=2, Sepal width=2, Petal length=2, Petal width=2. Double brackets create the required 2D structure of shape $(1, 4)$.
- `pred = model.predict(sample)[0]`: Predicts class array `[0]`; `[0]` extracts integer `0`.
- `iris.target_names[pred]`: Maps `0` to `"setosa"`.

Block 4: Comparative Scatter Subplots (Lines 35–60)

- `plt.subplot(1, 2, 1)`: Activates the left panel.
- `plt.scatter(x_test[:, 0], x_test[:, 1], c=y_prediction)`: Plots test points using Sepal length (col 0) on X-axis and Sepal width (col 1) on Y-axis. `c=y_prediction` colors points by the model's *predicted* class.
- `plt.subplot(1, 2, 2)`: Activates the right panel.
- `plt.scatter(x_test[:, 0], x_test[:, 1], c=y_test)`: Plots identical test points, but `c=y_test` colors points by their *actual ground-truth* species.
- Comparing the two panels allows the examiner to visually identify any misclassified flowers!
- `plt.show()`: Renders the figure window.

Section 6: Complete Function, Parameter and Syntax Reference

Source Code Coverage Checklist

Item in Source	Type	Library / Module	Verification Status
`load_iris()`	Function	`sklearn.datasets`	Confirmed present in source (Line 11)
`iris.data` / `iris.target`	Attributes	Scikit-Learn Bunch	Confirmed present in source (Lines 13-14)
`iris.target_names`	Attribute	Scikit-Learn Bunch	Confirmed present in source (Line 32)
`train_test_split()`	Function	`sklearn.model_selection`	Confirmed present in source (Line 16)
`test_size=0.25`	Parameter	`train_test_split`	Confirmed present in source (Line 17)
`KNeighborsClassifier()`	Class	`sklearn.neighbors`	Confirmed present in source (Line 20)
`n_neighbors=5`	Parameter	`KNeighborsClassifier`	Confirmed present in source (Line 20)
`.fit(X, y)`	Method	Classifier Estimator	Confirmed present in source (Line 22)
`.predict(X)`	Method	Classifier Estimator	Confirmed present in source (Lines 24, 30)
`metrics.accuracy_score()`	Function	`sklearn.metrics`	Confirmed present in source (Line 26)
`plt.subplot(1, 2, 1)`	Function	`matplotlib.pyplot`	Confirmed present in source (Lines 35, 48)
`plt.scatter()` with `c=`	Function	`matplotlib.pyplot`	Confirmed present in source (Lines 37, 50)
`plt.show()`	Function	`matplotlib.pyplot`	Confirmed present in source (Line 60)

Detailed 10-Point Breakdown for Key Functions

1. **`KNeighborsClassifier()`**
   1. **Exact Name:** `sklearn.neighbors.KNeighborsClassifier`
   2. **Module:** `sklearn.neighbors`
   3. **Purpose:** Classifier implementing the k-nearest neighbors vote.
   4. **General Syntax:** `model = KNeighborsClassifier(n_neighbors=5, weights='uniform', metric='minkowski', p=2)`
   5. **Parameters:** `n_neighbors`: Number of neighbors k to query; `weights`: 'uniform' (all neighbors equal) or 'distance' (closer neighbors have greater influence); `metric`: distance metric ('minkowski' with $p=2$ is Euclidean).
   6. **Small Example:** `knn = KNeighborsClassifier(n_neighbors=3).fit([[0], [1], [2]], [0, 0, 1])`
   7. **Expected Result:** Trained classifier ready for inference.
   8. **Role in Program:** Core machine learning estimator trained on `x_train` and evaluated on `x_test`.
   9. **Common Mistakes:** Misspelling as `KNeighborClassifier` (missing the 's') or typing `n_neighbor` without 's'.
   10. **Memory Tip:** "K-Neighbors-Classifier with n_neighbors=5."

2. **`train_test_split()`**
   1. **Exact Name:** `sklearn.model_selection.train_test_split`
   2. **Module:** `sklearn.model_selection`
   3. **Purpose:** Splits feature and target arrays into random train and test subsets.
   4. **General Syntax:** `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=None, train_size=None, random_state=None)`
   5. **Parameters:** `X, y`: Arrays to split; `test_size=0.25`: Proportion reserved for testing (25%).
   6. **Small Example:** `X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2)`
   7. **Expected Result:** Four arrays unpacked in exact order.
   8. **Role in Program:** Partitions 150 Iris samples into 112 training and 38 testing pairs.
   9. **Common Mistakes:** Unpacking in wrong order (e.g. putting `y_train` before `x_test`).
   10. **Memory Tip:** "Train-Test, Train-Test: X first, then y."

3. **`metrics.accuracy_score()`**
   1. **Exact Name:** `sklearn.metrics.accuracy_score`
   2. **Module:** `sklearn.metrics`
   3. **Purpose:** Calculates the fraction of correctly classified samples: $↑rac{ ext{Correct Predictions}}{ ext{Total Samples}}$.
   4. **General Syntax:** `accuracy_score(y_true, y_pred)`
   5. **Parameters:** `y_true`: Ground-truth target labels; `y_pred`: Predicted labels from model.
   6. **Small Example:** `accuracy_score([0, 1, 1], [0, 1, 0])` $\implies$ $2/3 □pprox 0.6667$.
   7. **Expected Result:** Float scalar between 0.0 (0%) and 1.0 (100%).
   8. **Role in Program:** Evaluates accuracy on the 38 test samples: `accuracy_score(y_test, y_prediction)`.
   9. **Common Mistakes:** Passing `x_test` instead of `y_test`.
   10. **Memory Tip:** "Truth first, Prediction second: (y_test, y_pred)."

Section 7: Step-by-Step Execution Walkthrough

Step	Program Line	Operation Applied	Array Shapes & Intermediate Values
1	11-14	Load Iris data	`x` shape: $(150, 4)$ floats; `y` shape: $(150,)$ integers ($0, 1, 2$)
2	16-18	Partition train/test	`x_train` : $(112, 4)$, `y_train` : $(112,)$ `x_test` : $(38, 4)$, `y_test` : $(38,)$
3	20	Instantiate model	`model` initialized with $k=5$, Euclidean metric
4	22	Train (fit) model	112 training points indexed in memory
5	24	Inference on test set	`y_prediction` shape: $(38,)$ predicted class labels
6	26	Accuracy score	Computes matches: e.g. $37 / 38 \implies \mathbf{0.97368}$
7	28-32	Classify new sample	`sample = [[2, 2, 2, 2]]` of shape $(1, 4) \implies$ predicts `0` (Setosa)
8	35-46	Render Subplot 1	Left panel: 38 points colored by `y_prediction`
9	48-59	Render Subplot 2	Right panel: 38 points colored by true `y_test`
10	60	Display window	GUI figure opens displaying both comparative scatter plots

Section 8: Output and Interpretation

Console Output (Representative Run)

```
Accuracy =  0.9736842105263158
New Sample belongs to  setosa
```

Output Breakdown & Graphical Interpretation

- **Accuracy Value (0.9737):** Indicates that out of 38 randomly selected test flowers, 37 flowers (97.37%) were classified correctly according to their true biological species. Because `random_state` is not fixed in `e05`, accuracy varies between 0.92 and 1.00 depending on the split.
- **New Sample Prediction ('setosa'):** The query coordinates $(2, 2, 2, 2)$ describe small petal lengths and widths (2.0 cm), characteristic of the *Iris setosa* cluster.
- **Side-by-Side Subplots:**
  - **X-axis:** Sepal length (ranging from 4.5 cm to 8.0 cm).
  - **Y-axis:** Sepal width (ranging from 2.0 cm to 4.5 cm).
  - **Left Plot (Predicted):** Shows the classes assigned by the KNN decision boundaries.
  - **Right Plot (Actual):** Shows the true ground truth species. A misclassified flower appears as a point with different colors between the left and right subplots.

Section 9: Exam Preparation

A. Practical-Record Answer Format

- **Aim:** To implement the K-Nearest Neighbors (KNN) algorithm on the Iris dataset, evaluate classification accuracy, predict an unseen sample, and visualize predicted vs. actual classifications using Matplotlib subplots.
- **Algorithm:** Reproduce the 15-step algorithm from Section 3.
- **Key Code Statements:**

```
iris = load_iris(); x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25)
model = KNeighborsClassifier(n_neighbors=5).fit(x_train, y_train)
y_prediction = model.predict(x_test)
print("Accuracy = ", metrics.accuracy_score(y_test, y_prediction))
pred = model.predict([[2, 2, 2, 2]])[0]
print("New Sample belongs to ", iris.target_names[pred])
```

- **Result:** KNN classifier achieved over 95% accuracy on the Iris test dataset, correctly classified sample $(2, 2, 2, 2)$ as Setosa, and generated side-by-side comparative subplots.

B. Question Bank (2, 3, and 5 Marks)

- **2 Marks:** What is KNN? Why is it called a lazy learner?
- **3 Marks:** Explain the role of `test_size=0.25` and `n_neighbors=5` in `e05_knn.py`.
- **5 Marks:** Explain the complete classification workflow of the KNN algorithm using a small numerical example with Euclidean distance calculations.

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Viva Q1: What type of machine learning algorithm is KNN?

Answer: KNN is a supervised, instance-based, non-parametric classification (and regression) algorithm.

Viva Q2: Why is KNN called a "lazy learner"?

Answer: Because it does not build an explicit model or mathematical function during training (`.fit()`); it merely memorizes the training dataset and delays all computation until prediction time.

Viva Q3: What distance metric does KNeighborsClassifier use by default?

Answer: It uses the Minkowski distance with parameter $p=2$, which is equivalent to standard Euclidean distance ($d = \sqrt{\sum (x_i - y_i)^2}$).

Viva Q4: What does the parameter n_neighbors=5 represent?

Answer: It specifies $k=5$, meaning the algorithm queries the 5 closest training points to determine the majority vote for class assignment.

Viva Q5: What happens if k is set too small (e.g., k=1)?

Answer: The model becomes overly sensitive to noise and outliers, creating jagged decision boundaries and causing overfitting (high variance).

Viva Q6: What happens if k is set too large (e.g., k=100)?

Answer: The model smooths out decision boundaries excessively, ignoring local structures and causing underfitting (high bias).

Viva Q7: What are the 4 features in the Iris dataset?

Answer: Sepal length, Sepal width, Petal length, and Petal width (all measured in centimeters).

Viva Q8: How many total samples and classes are in the Iris dataset?

Answer: 150 total samples divided equally into 3 species: Setosa (0), Versicolor (1), and Virginica (2) with 50 samples each.

Viva Q9: Why is sample defined as [[2, 2, 2, 2]] with double brackets?

Answer: Scikit-Learn's `.predict()` method expects a 2D array of shape `(n_samples, n_features)`. Double brackets create a 2D array of shape $(1, 4)$.

Viva Q10: What does test_size=0.25 mean in train_test_split?

Answer: It allocates 25% of the samples (38 samples) for evaluating test accuracy and 75% (112 samples) for training.

Viva Q11: How is accuracy_score calculated?

Answer: Accuracy is the ratio of correctly predicted samples to the total number of evaluated test samples: $ ext{Accuracy} = rac{ ext{Correct}}{ ext{Total}}$.

Viva Q12: Why are columns 0 and 1 labeled "Sepal length" and "Sepal width" in the scatter plot?

Answer: In `iris.data`, column index 0 corresponds to Sepal length and column index 1 corresponds to Sepal width.

Viva Q13: Can KNN be used for regression problems?

Answer: Yes. In KNN regression (`KNeighborsRegressor`), the target output is the average (mean) value of the k nearest neighbors rather than a majority vote.

Viva Q14: Why does feature scaling matter in KNN?

Answer: Because KNN relies on distance calculations. Features with larger numeric scales dominate the distance metric, making normalization/scaling critical when features have different units.

Viva Q15: What is the computational complexity of KNN during testing?

Answer: It is $O(N \cdot d)$ per query sample, where N is the number of training samples and d is the number of feature dimensions, making it slow on very large datasets.

Viva Q16: How do the two subplots help evaluate the model?

Answer: Subplot 1 displays predicted labels while Subplot 2 displays true labels. Any data point that changes color between the two plots reveals a misclassification.

Section 11: Common Mistakes and Troubleshooting

Common Mistake	Error Message / Symptom	Correction
Passing 1D list `[2, 2, 2, 2]` to `predict()`	`ValueError: Expected 2D array, got 1D array instead`	Use double brackets: `sample = [[2, 2, 2, 2]]`.
Misspelling `n_neighbors` as `n_neighbor`	`TypeError: __init__() got an unexpected keyword argument 'n_neighbor'`	Include the trailing 's': `KNeighborsClassifier(n_neighbors=5)`.
Swapping order in `train_test_split` unpacking	High loss of accuracy; test data fitted to train	Memorize exact order: `x_train, x_test, y_train, y_test`.
Mislabeled axis labels (e.g. calling columns 0 and 1 Petal)	Loss of practical exam marks for scientific inaccuracies	Label column 0 as Sepal length and column 1 as Sepal width.

Section 12: Memory Tricks & Last-Minute Rapid Revision

Master Mnemonic for KNN Code Flow:
"L-S-M-F-P-A-N-P"
Load Iris $ ightarrow$ **S**plit Train/Test $ ightarrow$ **M**odel KNeighbors(5) $ ightarrow$ **F**it Training Data $ ightarrow$ **P**redict Test Data $ ightarrow$ **A**ccuracy Score $ ightarrow$ **N**ew Sample [[2,2,2,2]] $ ightarrow$ **P**lot Subplots (Predicted vs Actual)

LAST-MINUTE EXAM SKELETON:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
import matplotlib.pyplot as plt

iris = load_iris()
x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25)

model = KNeighborsClassifier(n_neighbors=5).fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy =", metrics.accuracy_score(y_test, y_pred))
print("Sample species:", iris.target_names[model.predict([[2, 2, 2, 2]])[0]])

plt.subplot(1, 2, 1); plt.scatter(x_test[:, 0], x_test[:, 1], c=y_pred); plt.title("Predicted")
plt.subplot(1, 2, 2); plt.scatter(x_test[:, 0], x_test[:, 1], c=y_test); plt.title("Actual")
plt.show()
```

Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. Why is KNN called a "lazy learner" or instance-based algorithm?
Answer: Because it has no explicit training phase; it merely stores training instances in memory and defers all distance calculations until prediction time.
2. What is the effect of choosing a very small k ($k=1$) versus a large k?
Answer: A very small k ($k=1$) makes the model overly sensitive to noise/outliers (overfitting, high variance). A large k smooths boundaries too much (underfitting, high bias).
3. What distance metric does `KNeighborsClassifier` use by default?
Answer: Standard Euclidean distance ($p=2$ in Minkowski distance).
4. Why is feature scaling important for KNN in general?
Answer: Because Euclidean distance treats all coordinate axes equally; features with larger numerical magnitudes dominate the distance calculations unless standardized.
5. Why is `pred = model.predict(sample)[0]` followed by index `[0]`?
Answer: `predict()` returns a 1D array of predictions (e.g. `array([0])`); `[0]` extracts the scalar integer label to index `iris.target_names`.

13. Cross-Program Comparison Table (Programs 05 to 10)

The table below provides a rigorous cross-program comparative reference across all machine learning programs in the laboratory curriculum, highlighting learning paradigms, dataset characteristics, data splitting strategies, core algorithms, evaluation metrics, and diagnostic visualizations:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification on test samples
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	None (Structured terminal classification report)
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"`, `max_depth=4`, `min_samples_leaf=2`)	`classification_report()`	Full visual flowchart tree via `tree.plot_tree(filled=True)`
e08_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`train_size=0.8` (80:20 split)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score() * 100` (%)	None (Live interactive console input parsing with `float` & species mapping)
e09_kmeans.py	**Unsupervised Clustering**	Synthetic Blobs (300 × 2)	None (Full unlabeled dataset)	K-Means Clustering (`n_clusters=5`, `random_state=42`, `n_init="auto"`)	Inertia / Cluster assignments (`labels_`)	2D Scatter plot with `c=labels, cmap="viridis"` and centroids marked with red 'x'
e10_multiple_linear_regression.py	**Supervised Regression**	Diabetes Dataset (Features 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Mean Squared Error (MSE) & R^2 Score	Figure 1: Feature vs Target subplots; Figure 2: Actual vs Predicted fit line